07-Monad-확장-기능

개요

이 문서는 Monad 블록체인의 고유한 확장 기능들을 분석합니다. Monad는 이더리움 호환성을 유지하면서도 독자적인 기능들을 추가하여 성능과 기능성을 향상시켰습니다. 주요 확장 기능은 다음과 같습니다:

1. **Precompile 구현**: 표준 이더리움 precompile + BLS12-381 + Monad 커스텀 precompile
2. **스테이킹 메커니즘**: 네이티브 PoS 스테이킹 시스템
3. **시스템 트랜잭션**: 합의 레이어와 실행 레이어 간 통신
4. **Monad 전용 트랜잭션 처리**: 가스 계산 및 환불 정책 차이

구조

관련 파일 및 디렉토리

```
category/execution/
├── ethereum/
│ ├── precompiles.cpp # 표준 이더리움 precompile
│ └── precompiles_bls12.cpp # BLS12-381 암호화 precompile
└── monad/
├── monad_precompiles.cpp # Monad 커스텀 precompile
├── staking/
│ ├── staking_contract.hpp # 스테이킹 컨트랙트 인터페이스
```

```
| ├── staking_contract.cpp # 스테이킹 로직 구현
| └── util/ # 스테이킹 유틸리티
├── execute_system_transaction.cpp # 시스템 트랜잭션 실행
├── validate_system_transaction.cpp # 시스템 트랜잭션 검증
├── dispatch_transaction.cpp # 트랜잭션 디스패칭
└── monad_transaction_gas.cpp # Monad 가스 계산
```

1. Precompile 구현 분석

1.1 표준 이더리움 Precompile

Monad는 이더리움의 모든 표준 precompile을 지원합니다.

파일: `category/execution/ethereum/precompiles.cpp`

주요 함수: `resolve_precompile()`

```
template <Traits traits>
std::optional<PrecompiledContract> resolve_precompile(Address const  &address)
{
// 기본 precompile (모든 버전)
CASE(0x01, ecrecover_gas_cost<traits>, ecrecover_execute);
CASE(0x02, sha256_gas_cost<traits>, sha256_execute);
CASE(0x03, ripemd160_gas_cost<traits>, ripemd160_execute);
CASE(0x04, identity_gas_cost, identity_execute);
```

```
// Byzantium 이후 (EIP-196, EIP-197, EIP-198)
if constexpr (traits::evm_rev() >= EVMC_BYZANTIUM) {
CASE(0x05, expmod_gas_cost<traits>, expmod_execute);
CASE(0x06, ecadd_gas_cost<traits>, ecadd_execute);
CASE(0x07, ecmul_gas_cost<traits>, ecmul_execute);
CASE(0x08, snarkv_gas_cost<traits>, snarkv_execute);
}

// Istanbul 이후 (EIP-152)
if constexpr (traits::evm_rev() >= EVMC_ISTANBUL) {
CASE(0x09, blake2bf_gas_cost<traits>, blake2bf_execute);
}

// Cancun 이후 (EIP-4844)
if constexpr (traits::evm_rev() >= EVMC_CANCUN) {
CASE(0x0A, point_evaluation_gas_cost<traits>, point_evaluation_execute);
}

// Prague 이후 (EIP-2537: BLS12-381)
if constexpr (traits::evm_rev() >= EVMC_PRAGUE) {
CASE(0x0B, bls12_g1_add_gas_cost, bls12_g1_add_execute);
CASE(0x0C, bls12_g1_msm_gas_cost, bls12_g1_msm_execute);
CASE(0x0D, bls12_g2_add_gas_cost, bls12_g2_add_execute);
```

```
CASE(0x0E, bls12_g2_msm_gas_cost, bls12_g2_msm_execute);
CASE(0x0F, bls12_pairing_check_gas_cost, bls12_pairing_check_execute);
CASE(0x10, bls12_map_fp_to_g1_gas_cost, bls12_map_fp_to_g1_execute);
CASE(0x11, bls12_map_fp2_to_g2_gas_cost, bls12_map_fp2_to_g2_execute);
}

// Rollup precompile (EIP-7951)
if constexpr (traits::eip_7951_active()) {
CASE(0x0100, p256_verify_gas_cost, p256_verify_execute);
}

return std::nullopt;
}
```

Precompile 실행 흐름

```
template <Traits traits>
std::optional<evmc::Result> check_call_eth_precompile(evmc_message const &msg)
{
auto const maybe_precompile = resolve_precompile<traits>(msg.code_address);
if (!maybe_precompile) {
return std::nullopt;
}
```

```
// EIP-7702: delegation 처리
if constexpr (traits::evm_rev() >= EVMC_PRAGUE) {
auto const delegated = (msg.flags & EVMC_DELEGATED) != 0;
if (delegated) {
return evmc::Result{evmc_status_code::EVMC_SUCCESS, msg.gas};
}
}

auto const [gas_cost_func, execute_func] = *maybe_precompile;

// 가스 계산
byte_string_view const input{msg.input_data, msg.input_size};
uint64_t const cost = gas_cost_func(input);

if (std::cmp_less(msg.gas, cost)) {
return evmc::Result{evmc_status_code::EVMC_OUT_OF_GAS};
}

// 실행
auto const [status_code, output_buffer, output_size] = execute_func(input);
return evmc::Result{evmc_result{
.status_code = status_code,
```

```
.gas_left = (status_code == EVMC_SUCCESS)
? msg.gas - static_cast<int64_t>(cost)
: 0,
.output_data = output_buffer,
.output_size = output_size,
.release = evmc_free_result_memory,
}};
}
```

특징:

- 컴파일 타임 조건부 활성화 (`if constexpr`)
- EVM 버전별 precompile 지원
- 가스 계산과 실행 분리
- 표준 EVMC 인터페이스 준수

1.2 BLS12-381 Precompile (EIP-2537)

파일: `category/execution/ethereum/precompiles_bls12.cpp`

BLS12-381은 영지식 증명과 서명 집계에 사용되는 타원곡선 암호화입니다.

G1/G2 그룹 연산

```
template <typename Group>
PrecompileResult add(byte_string_view const  input)
{
if (input.size() != 2 * Group::encoded_size) {
```

```
return PrecompileResult::failure();
}

// 두 점 읽기
auto  const a = Group::read(input.data());
auto  const b = Group::read(input.data() + Group::encoded_size);

if (!a.has_value() || !b.has_value()) {
return PrecompileResult::failure();
}

// 아핀 좌표 → 사영 좌표 변환
typename Group::Point a_non_affine;
Group::from_affine(&a_non_affine, &*a);

// 점 덧셈
typename Group::Point result_non_affine;
Group::add(&result_non_affine, &a_non_affine, &*b);

// 사영 좌표 → 아핀 좌표 변환
typename Group::AffinePoint result;
Group::to_affine(&result, &result_non_affine);
```

```
// 결과 직렬화
auto *const output_buf = static_cast<uint8_t *>(std::malloc(Group::encoded_size)
Group::write(result, output_buf);

return {
.status_code = EVMC_SUCCESS,
.obuf = output_buf,
.output_size = Group::encoded_size,
};
}
```

Multi-Scalar Multiplication (MSM) with Pippenger 알고리즘

```
template <typename Group>
PrecompileResult msm_pippenger(byte_string_view const  input, uint64_t  const  k
{
auto affine_points = std::vector<typename Group::AffinePoint>{};
affine_points.reserve(k);

auto scalars = std::vector<blst_scalar>{};
scalars.reserve(k);

// 입력 파싱
```

```
static constexpr auto pair_size = Group::encoded_size + 32;
for (auto const *ptr = input.data(); ptr != end_ptr; ptr += pair_size) {
auto const affine_point = Group::read(ptr);
if (!affine_point.has_value() ||
!Group::affine_point_in_group(&*affine_point)) {
return PrecompileResult::failure();
}

if (Group::affine_point_is_inf(&*affine_point)) {
continue; // 무한원점 스킵
}

affine_points.emplace_back(*affine_point);
scalars.emplace_back(read_scalar(ptr + Group::encoded_size));
}

// Pippenger MSM 알고리즘 실행
auto const scratch_size = Group::msm_scratch_size(n_points);
auto const scratch = std::make_unique_for_overwrite<uint8_t[]>(scratch_size);

typename Group::Point result;
Group::msm(
&result,
```

```
affine_point_ptrs.data(),
n_points,
scalar_ptrs.data(),
256u,
reinterpret_cast<limb_t *>(scratch.get()));

return result;
}
```

Pairing Check

```
PrecompileResult pairing_check(byte_string_view const  input)
{
static  constexpr  auto pair_size = G1::encoded_size + G2::encoded_size;
auto  const k = input.size() / pair_size;

auto result = *blst_fp12_one(); // 단위원소로 초기화

for (auto  const *ptr = input.data(); ptr != end_ptr; ptr += pair_size) {
auto  const maybe_g1 = G1::read(ptr);
auto  const maybe_g2 = G2::read(ptr + G1::encoded_size);

// Miller loop 계산
blst_fp12 paired;
```

```
blst_miller_loop(&paired, &*maybe_g2, &*maybe_g1);
blst_fp12_mul(&result, &result, &paired);
}

// Final exponentiation
blst_final_exp(&result, &result);

// 결과가 1인지 확인
bool const is_valid = blst_fp12_is_one(&result);
output_buf[31] = is_valid ? 1 : 0;

return {.status_code = EVMC_SUCCESS, .obuf = output_buf, .output_size = 32};
}
```

BLS12-381 최적화:

- BLST 라이브러리 사용 (하드웨어 가속)
- Pippenger 알고리즘으로 MSM 최적화
- 가스 비용 할인 테이블 (k개 연산 시 할인율 적용)

```
template <>
uint16_t msm_discount<G1>(uint64_t const k)
{
static constexpr auto table = std::array<uint16_t, 128>{
```

```
1000, 949, 848, 797, 764, 750, 738, 728, 719, 712, ...
};
return  table[std::min(k, 128ul) - 1];
}
```

1.3 Monad 커스텀 Precompile

파일: `category/execution/monad/monad_precompiles.cpp`

Monad는 스테이킹 컨트랙트를 precompile로 구현하여 성능을 최적화했습니다.

Precompile 디스패칭

```
template <Traits traits>
std::optional<evmc::Result> check_call_monad_precompile(
State &state, CallTracerBase &call_tracer, evmc_message const  &msg)
{
if  constexpr (traits::monad_rev() < MONAD_FOUR) {
return std::nullopt; // MONAD_FOUR 이전에는 비활성화
}

if (msg.code_address != staking::STAKING_CA) {
return std::nullopt; // 스테이킹 주소가 아니면 패스
}

// CALL만 허용 (DELEGATECALL, STATICCALL 등 불가)
```

```
if (msg.kind != EVMC_CALL || msg.flags != 0) {
return evmc::Result{evmc_status_code::EVMC_REJECTED};
}

// 가스 계산 및 메서드 디스패칭
byte_string_view input{msg.input_data, msg.input_size};
auto const [method, cost] = staking::StakingContract::precompile_dispatch(input
if (std::cmp_less(msg.gas, cost)) {
return evmc::Result{evmc_status_code::EVMC_OUT_OF_GAS};
}

// 스테이킹 컨트랙트 실행
staking::StakingContract contract(state, call_tracer);
auto const res = (contract.*method)(input, msg.sender, msg.value);
if (res.has_value()) {
int64_t const gas_left = msg.gas - static_cast<int64_t>(cost);
return evmc::Result(EVMC_SUCCESS, gas_left, 0,
res.value().data(), res.value().size());
}
return evmc::Result(EVMC_REVERT, 0, 0,
res.error().message().data(),
res.error().message().size());
```

```
}
```

Precompile 통합

```
template <Traits traits>
bool  is_precompile(Address const  &address)
{
return  is_eth_precompile<traits>(address) ||
(address == staking::STAKING_CA);
}

template <Traits traits>
std::optional<evmc::Result> check_call_precompile(
State &state, CallTracerBase &call_tracer, evmc_message const  &msg)
{
// 먼저 이더리움 precompile 확인
if (auto maybe_result = check_call_eth_precompile<traits>(msg)) {
return maybe_result;
}

// Monad precompile 확인
return  check_call_monad_precompile<traits>(state, call_tracer, msg);
}
```

특징:

- 스테이킹 컨트랙트 주소: `staking::STAKING_CA`
- MONAD_FOUR 포크부터 활성화
- 네이티브 코드로 실행되어 가스 효율적
- 상태 접근 및 이벤트 발생 가능

2. 스테이킹 메커니즘 분석

2.1 스테이킹 컨트랙트 구조

파일: `category/execution/monad/staking/staking_contract.hpp`

Monad의 스테이킹은 네이티브 PoS 시스템으로, 검증자(Validator)와 위임자(Delegator) 모델을 사용합니다.

스토리지 변수

```
class StakingContract::Variables

{

State &state_;


// 단일 슬롯 상수 (네임스페이스 0x0)

StorageVariable<u64_be> epoch; // 현재 에포크

StorageVariable<bool> in_epoch_delay_period; // 에포크 경계 기간 여부

StorageVariable<u64_be> last_val_id; // 마지막 검증자 ID


// 검증자 세트 (네임스페이스 0x1, 0x2, 0x3)
```

```
StorageArray<u64_be> valset_execution; // 실시간 실행 검증자 세트

StorageArray<u64_be> valset_consensus; // 합의 검증자 세트 (스냅샷)

StorageArray<u64_be> valset_snapshot; // 이전 에포크 스냅샷



// 매핑 (각 네임스페이스 0x04 ~ 0x0C)

StorageVariable<u64_be> val_id(Address const  &secp_eth_address);

StorageVariable<u64_be> val_id_bls(Address const  &bls_eth_address);

StorageVariable<u256_be> val_bitset_bucket(u64_be const  val_id);

ValExecution val_execution(u64_be const  id);

ConsensusView consensus_view(u64_be const  id);

SnapshotView snapshot_view(u64_be const  id);

Delegator delegator(u64_be const  val_id, Address const  &address);

StorageVariable<WithdrawalRequest> withdrawal_request(...);

StorageVariable<RefCountedAccumulator> accumulated_reward_per_token(...);

};
```

네임스페이스 구조:

- `0x00` : 전역 상수 (epoch, in_boundary, last_val_id)
- `0x01-0x03` : 검증자 세트 배열
- `0x04-0x0C` : 매핑 (각 매핑이 전체 네임스페이스 소유)

검증자 정보 구조

```
struct ValExecution {
```

```
StorageVariable<u256_be> stake; // 총 스테이크

StorageVariable<u256_be> commission; // 수수료율

StorageVariable<u256_be> accumulated_reward_per_token; // 누적 보상

StorageVariable<u256_be> unclaimed_rewards; // 미청구 보상

StorageVariable<u64_be> flags; // 상태 플래그

StorageVariable<Address> secp_pubkey; // SECP256k1 공개키

StorageVariable<Address> bls_pubkey; // BLS 공개키

// ... 기타 필드

};
```

위임자 정보 구조

```
struct Delegator {

StorageVariable<u256_be> stake; // 활성 스테이크

StorageVariable<u256_be> delta_stake; // 다음 에포크 델타

StorageVariable<u256_be> next_delta_stake; // 경계 후 델타

StorageVariable<u256_be> rewards; // 누적 보상

StorageVariable<u256_be> accumulated_reward_per_token; // 마지막 체크 시점

StorageVariable<u64_be> delta_epoch; // 델타 활성화 에포크

StorageVariable<u64_be> next_delta_epoch; // 다음 델타 에포크

// 연결 리스트 포인터

StorageVariable<Address> next_delegator;

StorageVariable<u64_be> next_validator;
```

```
};
```

2.2 스테이킹 작업 흐름

검증자 추가 (Add Validator)

```
Result<byte_string> StakingContract::precompile_add_validator(
byte_string_view input, evmc_address const  &sender, evmc_uint256be const  &valu
{
// 1. 서명 검증 (SECP256k1 + BLS)
auto  const secp_pubkey = recover_secp256k1_pubkey(secp_sig, msg_hash);
auto  const bls_valid = verify_bls_signature(bls_pubkey, bls_sig, msg_hash);

// 2. 검증자 ID 할당
auto  const val_id = vars.last_val_id.load().native() + 1;
vars.last_val_id.store(val_id);

// 3. 검증자 정보 저장
auto val = vars.val_execution(val_id);
val.secp_pubkey().store(secp_pubkey);
val.bls_pubkey().store(bls_pubkey);
val.commission().store(commission);
val.stake().store(value.native());
```

```
// 4. 매핑 업데이트
vars.val_id(secp_eth_address).store(val_id);
vars.val_id_bls(bls_eth_address).store(val_id);


// 5. 검증자 세트에 추가
add_to_valset(val_id);
vars.valset_execution.push(val_id);


// 6. 초기 위임 (자기 자신)
delegate(val_id, value.native(), sender);


emit_validator_created_event(val_id, sender, commission);
return  abi_encode_uint(val_id);
}
```

가스 비용: 505,125 gas

- EC_RECOVER: 3,000 gas
- BLS_VERIFY: 150,000 gas
- 스토리지 작업: 352,125 gas

위임 (Delegate)

```
Result<void> StakingContract::delegate(
u64_be val_id, uint256_t const  &amount, Address const  &delegator_addr)
{
```

```
auto del = vars.delegator(val_id, delegator_addr);

auto val = vars.val_execution(val_id);


// 1. 위임자 최신 상태로 업데이트 (보상 계산)

pull_delegator_up_to_date(val_id, del);


// 2. 활성화 에포크 결정

uint64_t  const activation_epoch = get_activation_epoch();


// 3. 델타 스테이크 업데이트

if (del.get_delta_epoch().native() == 0) {

// 첫 델타

del.delta_stake().store(amount);

del.set_delta_epoch(activation_epoch);

} else  if (del.get_next_delta_epoch().native() == 0) {

// 두 번째 델타 (경계 기간)

del.next_delta_stake().store(amount);

del.set_next_delta_epoch(activation_epoch);

} else {

// 기존 델타에 추가

auto  const old_delta = del.delta_stake().load().native();

del.delta_stake().store(checked_add(old_delta, amount));

}
```

```
// 4. 검증자 총 스테이크 업데이트
auto const old_stake = val.stake().load().native();
val.stake().store(checked_add(old_stake, amount));

// 5. 미래 누적기 참조 카운트 증가
increment_accumulator_refcount(val_id);

// 6. 연결 리스트 업데이트
linked_list_insert(val_id, delegator_addr);

emit_delegation_event(val_id, delegator_addr, amount, activation_epoch);
return success();
}
```

가스 비용: 260,850 gas

위임 해제 (Undelegate)

```
Result<byte_string> StakingContract::precompile_undelegate(
byte_string_view input, evmc_address const &sender, evmc_uint256be const &valu
{
auto del = vars.delegator(val_id, sender);
```

```
// 1. 최신 상태로 업데이트
pull_delegator_up_to_date(val_id, del);

// 2. 충분한 스테이크 확인
auto  const current_stake = del.stake().load().native();
if (current_stake < amount) {
return StakingError::InsufficientStake;
}

// 3. 출금 요청 생성
auto withdrawal = vars.withdrawal_request(val_id, sender, withdrawal_id);
withdrawal.store(WithdrawalRequest{
.amount = amount,
.acc = del.accumulated_reward_per_token().load(),
.epoch = get_activation_epoch()
});

// 4. 스테이크 감소
del.stake().store(checked_sub(current_stake, amount));

// 5. 검증자 총 스테이크 감소
auto val = vars.val_execution(val_id);
auto  const val_stake = val.stake().load().native();
```

```
val.stake().store(checked_sub(val_stake, amount));


emit_undelegate_event(val_id, sender, withdrawal_id, amount, activation_epoch);
return  abi_encode_uint(withdrawal_id);
}
```

가스 비용: 147,750 gas

2.3 보상 계산 메커니즘

Monad는 **누적 보상 방식(Accumulator Pattern)**을 사용합니다.

보상 적용

```
Result<void> StakingContract::apply_reward(
u64_be const  val_id, Address const  &from,
uint256_t const  &new_rewards, uint256_t const  &active_stake)
{
// 1. 토큰당 보상 증가량 계산
auto  const reward_acc = checked_mul_div(new_rewards, UNIT_BIAS, active_stake);


// 2. 누적기에 추가
auto val_execution = vars.val_execution(val_id);
auto  const acc = checked_add(
val_execution.accumulated_reward_per_token().load().native(),
reward_acc);
```

```
val_execution.accumulated_reward_per_token().store(acc);


// 3. 미청구 보상 증가
auto const unclaimed_rewards = checked_add(
val_execution.unclaimed_rewards().load().native(),
new_rewards);
val_execution.unclaimed_rewards().store(unclaimed_rewards);


emit_validator_rewarded_event(val_id, from, new_rewards);
return success();
}
```

보상 청구

```
Result<uint256_t> StakingContract::apply_compound(
u64_be const val_id, Delegator &del)
{
// 1. 에포크 누적기 값 가져오기 (참조 카운트 감소)
auto const epoch_acc = decrement_accumulator_refcount(del.get_delta_epoch(), va
// 2. 보상 계산
auto const stake = del.stake().load().native();
auto const acc = del.accumulated_reward_per_token().load().native();
auto const rewards = calculate_rewards(stake, epoch_acc.native(), acc);
```

```
// 3. 누적기 업데이트
del.accumulated_reward_per_token().store(epoch_acc);

// 4. 델타 스테이크를 활성 스테이크로 병합
auto const delta_stake = del.delta_stake().load().native();
auto const compounded_stake = checked_add(stake, delta_stake);
del.stake().store(compounded_stake);

// 5. 델타 승격
promote_delta(del);

return rewards;
}

Result<uint256_t> calculate_rewards(
uint256_t const &stake,
uint256_t const &current_acc,
uint256_t const &last_checked_acc)
{
auto const delta = checked_sub(current_acc, last_checked_acc);
return checked_mul_div(delta, stake, UNIT_BIAS);
```

```
}
```

보상 계산 공식:

```
rewards = (current_acc - last_checked_acc) * stake / UNIT_BIAS
```

여기서:

- `current_acc` : 현재 누적 보상
- `last_checked_acc` : 마지막 체크 시점의 누적 보상
- `stake` : 위임자의 스테이크
- `UNIT_BIAS` : 정밀도 보정 상수 (10^18)

2.4 에포크 관리

에포크 변경 시스템 콜

```
Result<void> StakingContract::syscall_on_epoch_change(
byte_string_view input, uint256_t const  &value)
{
auto  const old_epoch = vars.epoch.load();
auto  const new_epoch = old_epoch.native() + 1;

// 1. 에포크 증가
vars.epoch.store(new_epoch);

// 2. 경계 기간 종료
```

```
vars.in_epoch_delay_period.store(false);


// 3. 스냅샷을 합의 세트로 복사
auto const consensus_size = vars.valset_consensus.size();
for (size_t i = 0; i < consensus_size; ++i) {
auto const val_id = vars.valset_consensus[i].load();
vars.valset_snapshot[i].store(val_id);
}


emit_epoch_changed_event(old_epoch, new_epoch);
return success();
}
```

스냅샷 시스템 콜

```
Result<void> StakingContract::syscall_snapshot(
byte_string_view input, uint256_t const &value)
{
// 1. 경계 기간 시작
vars.in_epoch_delay_period.store(true);


// 2. 실행 세트를 합의 세트로 복사 (상위 N개)
auto const execution_size = vars.valset_execution.size();
```

```
std::vector<std::pair<u64_be, u256_be>> validators;

for (size_t i = 0; i < execution_size; ++i) {

auto const val_id = vars.valset_execution[i].load();

auto const stake = vars.val_execution(val_id).stake().load();

validators.push_back({val_id, stake});

}


// 3. 스테이크 기준 정렬

std::sort(validators.begin(), validators.end(),

[](auto const &a, auto const &b) { return a.second > b.second; });


// 4. 상위 N개를 합의 세트로 설정

size_t const max_validators = std::min(validators.size(), MAX_CONSENSUS_VALIDAT

for (size_t i = 0; i < max_validators; ++i) {

vars.valset_consensus[i].store(validators[i].first);

}


return success();

}
```

에포크 타임라인:

```
Epoch N Boundary Period Epoch N+1

|----------------|-------------------|----------------|
```

```
실행 세트 변경 스냅샷 생성 에포크 변경

(실시간) (상위 N개 선택) (스냅샷 활성화)
```

3. 시스템 트랜잭션 분석

3.1 시스템 트랜잭션 개념

시스템 트랜잭션은 합의 레이어(Consensus)가 실행 레이어(Execution)에 상태 변경을 지시하기 위한 특수 트랜잭션입니다.

파일: `category/execution/monad/execute_system_transaction.cpp`

특징:

- 발신자: `SYSTEM_SENDER` (특수 주소)
- 가스 리밋: 0 (가스 소비 없음)
- 타입: Legacy 트랜잭션만 허용
- 대상: 스테이킹 컨트랙트만 호출 가능
- 실패 불가: 항상 성공 반환

3.2 시스템 트랜잭션 검증

파일: `category/execution/monad/validate_system_transaction.cpp`

```
template <Traits traits>

Result<void> static_validate_system_transaction(

Transaction const  &tx, Address const  &sender)

{

// MONAD_FOUR 포크 이전에는 불가
```

```
if  constexpr (traits::monad_rev() < MONAD_FOUR) {
return SystemTransactionError::SystemTxnBeforeFork;
}

// 발신자 확인
if (sender != SYSTEM_SENDER) {
return SystemTransactionError::BadSender;
}

// 트랜잭션 타입 확인
if (tx.type != TransactionType::legacy) {
return SystemTransactionError::TypeNotLegacy;
}

// 수신자 확인
if (!tx.to.has_value() || tx.to != staking::STAKING_CA) {
return SystemTransactionError::InvalidSystemContract;
}

// 가스 관련 필드 확인
if (tx.gas_limit != 0 ||
tx.max_fee_per_gas != 0 ||
tx.max_priority_fee_per_gas != 0) {
```

```
return SystemTransactionError::GasNonZero;
}

// EIP-7702 권한 리스트 확인
if (!tx.authorization_list.empty()) {
return SystemTransactionError::NonEmptyAuthorizationList;
}

return  success();
}
```

3.3 시스템 트랜잭션 실행

```
template <Traits traits>
Result<Receipt> ExecuteSystemTransaction<traits>::operator()()
{
// 1. 정적 검증
BOOST_OUTCOME_TRY(static_validate_system_transaction<traits>(tx_, sender_));

// 2. 상태 생성
State state{block_state_, Incarnation{header_.number, i_ + 1}};
state.set_original_nonce(sender_, tx_.nonce);
```

```
// 3. 실행
auto result = execute(state);


// 4. 병합 시도
if (block_state_.can_merge(state)) {
if (result.has_error()) {
return std::move(result.error());
}
auto  const receipt = execute_final(state);
block_state_.merge(state);
return receipt;
}


// 5. 재시도 (충돌 발생 시)
block_metrics_.inc_retries();
State retry_state{block_state_, Incarnation{header_.number, i_ + 1}};
result = execute(retry_state);
auto  const receipt = execute_final(retry_state);
block_state_.merge(retry_state);
return receipt;
}
```

3.4 시스템 콜 디스패칭

```
Result<void> ExecuteSystemTransaction<traits>::execute_staking_syscall(
State &state, byte_string_view calldata, uint256_t const  &value)
{
// 스테이킹 계정 생성 (존재하지 않으면)
state.add_to_balance(staking::STAKING_CA, 0);


staking::StakingContract contract(state, call_tracer_);
// 함수 셀렉터 파싱
auto  const signature = intx::be::unsafe::load<uint32_t>(calldata.substr(0, 4).d
calldata.remove_prefix(4);


switch (signature) {
case SyscallSelector::REWARD:
return  contract.syscall_reward(calldata, value);
case SyscallSelector::SNAPSHOT:
return  contract.syscall_snapshot(calldata, value);
case SyscallSelector::ON_EPOCH_CHANGE:
return  contract.syscall_on_epoch_change(calldata, value);
}
return staking::StakingError::MethodNotSupported;
}
```

시스템 콜 함수 셀렉터:

- `syscallReward(address)` : 0x791bdcf3
- `syscallSnapshot()` : 0x157eeb21
- `syscallOnEpochChange(uint64)` : 0x1d4e9f02

4. Monad 전용 트랜잭션 처리 분석

4.1 트랜잭션 디스패칭

파일: `category/execution/monad/dispatch_transaction.cpp`

```
template <Traits traits>
Result<Receipt> dispatch_transaction(
Chain const &chain, uint64_t const i, Transaction const &transaction,
Address const &sender, /* ... */)
{
// 시스템 트랜잭션 확인
if (traits::monad_rev() >= MONAD_FOUR && sender == SYSTEM_SENDER) {
return ExecuteSystemTransaction<traits>{
chain, i, transaction, sender, header,
block_state, block_metrics, prev, call_tracer}();
}
// 일반 트랜잭션
return ExecuteTransaction<traits>{
chain, i, transaction, sender, authorities, header,
```

```
block_hash_buffer, block_state, block_metrics, prev,
call_tracer, state_tracer, revert_transaction}();
}
```

4.2 가스 환불 정책

파일: `category/execution/monad/monad_transaction_gas.cpp`

Monad는 이더리움과 다른 가스 환불 정책을 사용합니다.

```
template <Traits traits>
uint64_t compute_gas_refund(
Transaction const &tx, uint64_t const gas_remaining, uint64_t const refund)
{
if constexpr (traits::monad_rev() >= MONAD_FIVE) {
// MONAD_FIVE: 전체 환불 허용
return refund;
}
else if constexpr (traits::monad_rev() >= MONAD_ONE) {
// MONAD_ONE ~ MONAD_FOUR: 환불 없음
return 0;
}

// 이더리움 호환 모드: g* 계산
return g_star<traits>(tx, gas_remaining, refund);
```

```
}
```

이더리움 g 계산:

```
template <Traits traits>
uint64_t g_star(Transaction const &tx, uint64_t const gas_remaining, uint64_
{
uint64_t const gas_used = tx.gas_limit - gas_remaining;
if constexpr (traits::evm_rev() >= EVMC_LONDON) {
// EIP-3529: 최대 환불 = gas_used / 5
return std::min(refund, gas_used / 5);
}
else {
// 이전: 최대 환불 = gas_used / 2
return std::min(refund, gas_used / 2);
}
}
```

환불 가격 계산

```
template <Traits traits>
uint256_t refund_gas_price(Transaction const &tx, uint256_t const &base_fee_pe
{
```

```
if constexpr (traits::monad_rev() >= MONAD_FIVE) {
// MONAD_FIVE: 최소 기본 수수료로 환불
return monad_min_base_fee_wei(traits::monad_rev());
}

// 이더리움 호환: 실제 가스 가격으로 환불
return gas_price<traits>(tx, base_fee_per_gas);
}
```

가스 환불 정책 비교:

| 버전 | 환불 정책 | 환불 가격 |

|------|----------|----------|

| 이더리움 | gas_used / 5 (London 이후) | 실제 가스 가격 |

| MONAD_ONE ~ MONAD_FOUR | 환불 없음 | N/A |

| MONAD_FIVE | 전체 환불 | 최소 기본 수수료 |

4.3 가스 계산 차이점

스토리지 환불 (SSTORE):

이더리움:

```
- 0 → 0: 100 gas (warm), 2100 gas (cold)
- 0 → X: 20000 gas (cold), 환불 없음
- X → 0: 2900 gas, 환불 4800 gas
- X → Y: 2900 gas (warm)
```

Monad (MONAD_FIVE):

```
- 동일한 가스 비용
- 환불 제한 없음 (전체 환불)
- 최소 기본 수수료로 환불
```

트랜잭션 가스 계산:

```
uint64_t intrinsic_gas = 21000; // 기본 트랜잭션 비용


// 데이터 비용
for (auto byte : tx.data) {
intrinsic_gas += (byte == 0) ? 4 : 16;
}


// 접근 리스트 비용 (EIP-2930)
intrinsic_gas += tx.access_list.size() * 2400;
for (auto  const &entry : tx.access_list) {
intrinsic_gas += entry.storage_keys.size() * 1900;
}


// 컨트랙트 생성 비용
if (!tx.to.has_value()) {
intrinsic_gas += 32000;
```

```
if (traits::evm_rev() >= EVMC_SHANGHAI) {
intrinsic_gas += initcode_cost(tx.data.size());
}
}
```

최적화 기법

1. Precompile 최적화

네이티브 구현:

- C++로 직접 구현하여 EVM 오버헤드 제거
- 하드웨어 가속 활용 (BLST, AVX2)
- 컴파일 타임 최적화

가스 비용 최적화:

- 실제 연산 비용 반영
- MSM 할인 테이블 적용
- 배치 연산 지원

2. 스테이킹 최적화

스토리지 레이아웃:

- 네임스페이스 기반 매핑으로 충돌 방지
- 비트셋으로 검증자 존재 확인 (256개/슬롯)
- 연결 리스트로 동적 크기 지원

가스 비용 사전 계산:

```
constexpr  uint64_t ADD_VALIDATOR_OP_COST = compute_costs(OpCount{
.warm_sloads = 21,
.cold_sloads = 22,
.warm_sstores = 6,
.warm_sstore_nonzero = 15,
.cold_sstores = 9,
.events = 3,
.transfers = 0
}) + EC_RECOVER_COST + BLS_VERIFY_COST;
```

누적 보상 방식:

- O(1) 보상 계산
- 위임자별 개별 추적 불필요
- 에포크 경계 처리 최적화

3. 시스템 트랜잭션 최적화

가스 소비 없음:

- 합의 레이어가 실행 비용 부담
- 사용자 트랜잭션과 분리

병렬 실행 지원:

- 일반 트랜잭션과 동일한 충돌 감지
- 재시도 메커니즘 적용

참고 자료

이더리움 EIP

- EIP-152: BLAKE2b precompile
- EIP-196/197: bn256 precompile
- EIP-198: modexp precompile
- EIP-2537: BLS12-381 precompile
- EIP-3529: 가스 환불 감소
- EIP-4844: Blob 트랜잭션
- EIP-7702: 계정 위임

Monad 사양

- Monad Specification §4.2: Storage Gas Cost and Refunds
- MONAD_FOUR: 시스템 트랜잭션 도입
- MONAD_FIVE: 가스 환불 정책 변경

암호화 라이브러리

- BLST: BLS12-381 구현
- silkpre: Precompile 라이브러리
- SECP256k1: 서명 검증

코드 예시

Precompile 호출 예시

```
// 이더리움 표준 precompile
address ecrecover_addr = address(0x01);
bytes memory input = abi.encodePacked(hash, v, r, s);
(bool success, bytes memory result) = ecrecover_addr.call(input);


// BLS12-381 G1 덧셈
address bls_g1_add = address(0x0B);
bytes memory points = abi.encodePacked(point1, point2);
(bool success, bytes memory sum) = bls_g1_add.call(points);


// Monad 스테이킹 precompile
address staking = address(0x...); // STAKING_CA
bytes memory data = abi.encodeWithSignature("delegate(uint64)", validatorId);
(bool success, ) = staking.call{value: amount}(data);
```

스테이킹 워크플로우

```
// 1. 검증자 추가
bytes memory secpSig = ...;
bytes memory blsSig = ...;
bytes memory blsPubkey = ...;
uint256 commission = 10 * 10**16; // 10%
```

```
staking.call{value: minStake}(
abi.encodeWithSignature(
"addValidator(bytes,bytes,bytes)",
secpSig, blsSig, blsPubkey
)
);


// 2. 위임
uint64 validatorId = 1;
staking.call{value: delegateAmount}(
abi.encodeWithSignature("delegate(uint64)", validatorId)
);


// 3. 보상 청구
staking.call(
abi.encodeWithSignature("claimRewards(uint64)", validatorId)
);


// 4. 위임 해제
uint256 amount = 100 ether;
uint8 withdrawalId = 0;
staking.call(
abi.encodeWithSignature(
```

```
"undelegate(uint64,uint256,uint8)",
validatorId, amount, withdrawalId
)
);

// 5. 출금 (언락 기간 후)
staking.call(
abi.encodeWithSignature(
"withdraw(uint64,uint8)",
validatorId, withdrawalId
)
);
```

결론

Monad의 확장 기능들은 이더리움 호환성을 유지하면서도 성능과 기능성을 크게 향상시켰습니다:

1. **Precompile**: 표준 + BLS12-381 + 커스텀 스테이킹
2. **스테이킹**: 네이티브 PoS, 누적 보상 방식, O(1) 계산
3. **시스템 트랜잭션**: 합의-실행 통신, 가스 소비 없음
4. **가스 정책**: MONAD_FIVE에서 전체 환불 지원

이러한 기능들은 모두 네이티브 코드로 구현되어 최적의 성능을 제공하며, 명확한 인터페이스와 검증 로직으로 안전성을 보장합니다.